



# Departamento de Informática e Matemática Aplicada – DIMAp

## Comunicação HTTP

Prof. Nélio Cacho

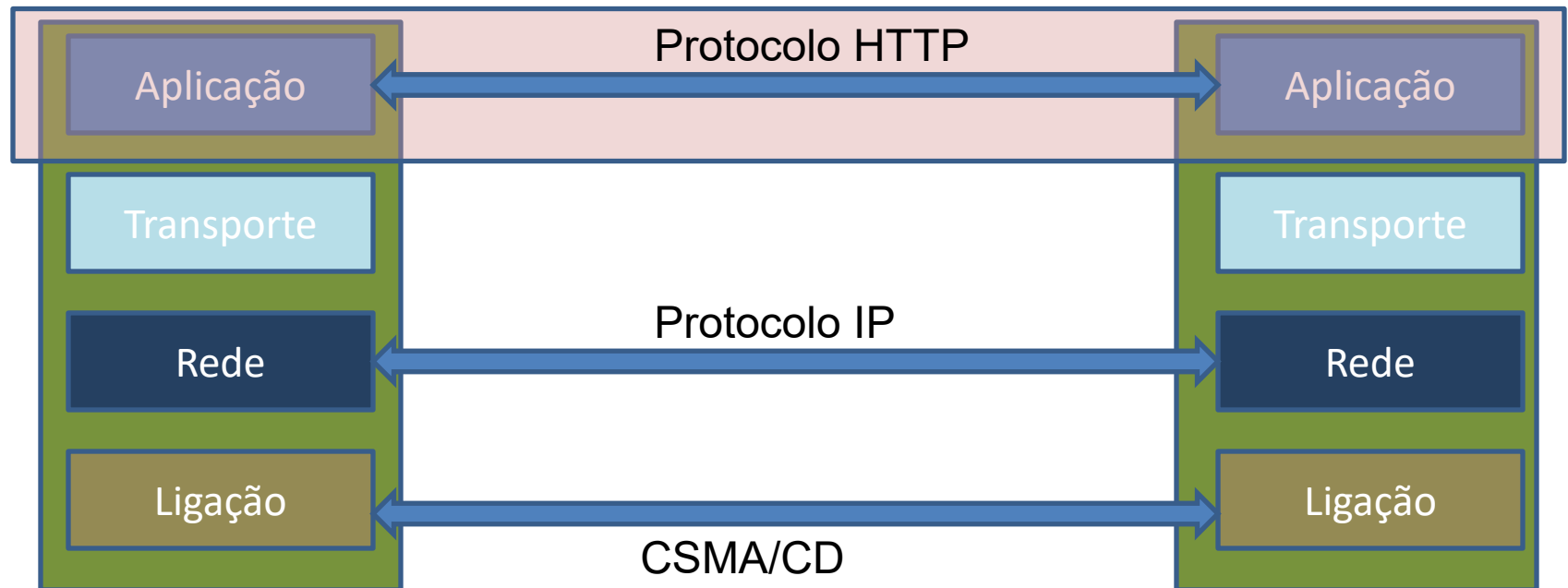
DIM0614 – Programação Distribuída

## Hora de silenciar o celular...



- Manter o telefone celular sempre desligado/silencioso quando estiver em sala de aula;
- Nunca atender o celular na sala de aula.

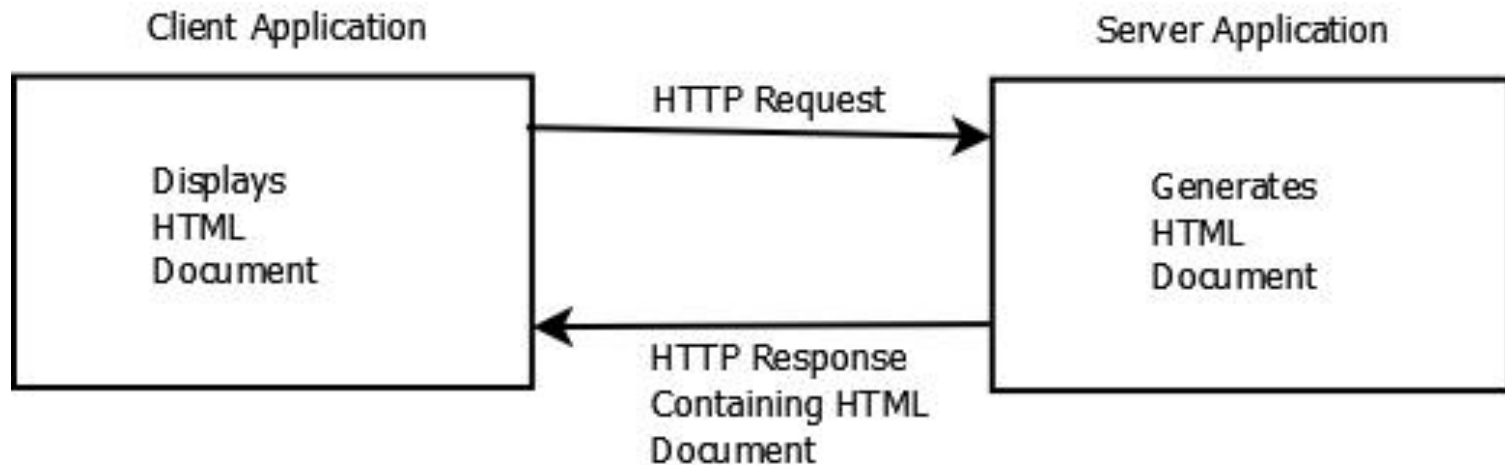
# Como realizar a comunicação ?



# Protocolo HTTP

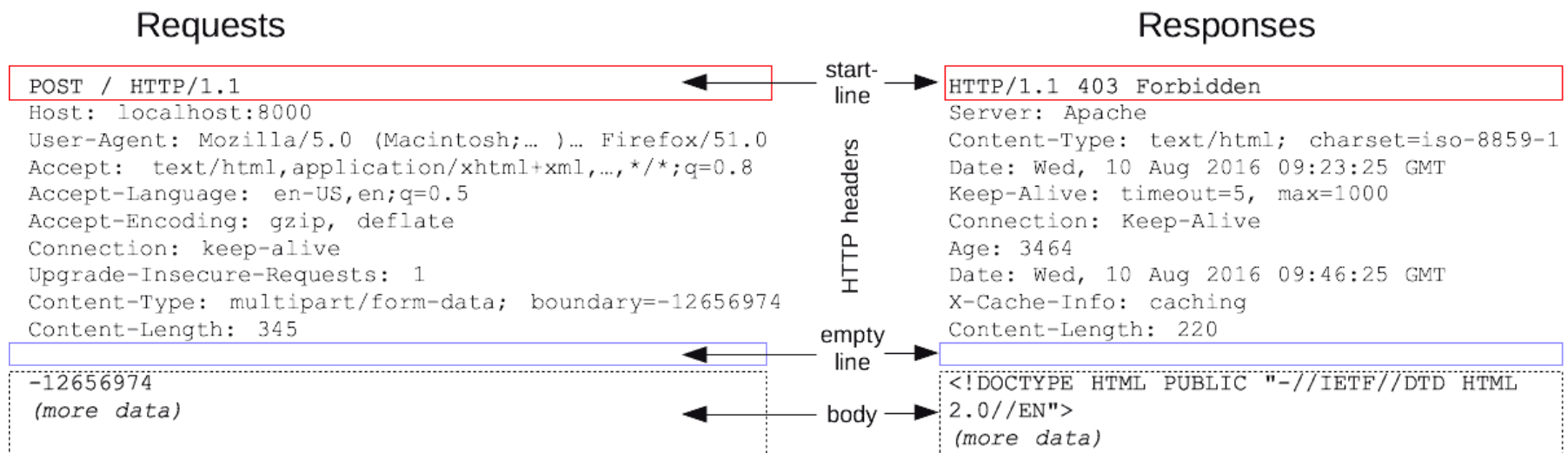
- O HTTP é um protocolo de rede utilizado para entregar recursos na World Wide Web (WWW).
- A maioria das pessoas usa um navegador para se comunicar na WWW.
- O navegador é uma aplicação cliente, enquanto o servidor web responde às solicitações do cliente.
- A porta padrão utilizada por esses servidores é a 80.

# Protocollo HTTP



# Mensagem HTTP

- Solicitações e respostas de mensagens HTTP utilizam esta estrutura:
  1. Uma linha indicando o tipo de mensagem;
  2. Zero ou mais linhas de cabeçalho;
  3. Uma linha em branco;
  4. Um corpo de mensagem opcional contendo dados.



# Formato da Linha de Solicitação Inicial

- O formato das linhas iniciais de solicitação e resposta diferem. A linha de solicitação consiste em três partes separadas por espaços:
  - Nome do método de solicitação;
  - Caminho local do recurso desejado: descreve o recurso a ser acessado. Segue o nome do hostname na URL de requisição;
  - Versão HTTP

Exe.: **GET /index HTTP/1.0**

# Request Method Name

- Os nomes dos métodos referem-se à ação solicitada pelo cliente. Eles são sempre escritos em letras maiúsculas.  
Métodos do  
HTTP/1.1:(<http://www.w3.org/Protocols/rfc2616/rfc2616-sec9.html>).
  - GET: Recupera informações identificadas pelo URI da solicitação.
  - POST: Insere e atualiza dados.
  - PUT: Armazena a entidade sob o URI fornecido.
  - DELETE: Solicita que o servidor apague o recurso identificado.

# Formato da Linha de Resposta Inicial

- A linha de resposta inicial tem três partes separadas por espaços:

1. Versão HTTP;

2. Código de status de resposta (três dígitos):

1. 1xx: Informativo.

2. 2xx: Sucesso.

3. 3xx: Redirecionamento.

4. 4xx: Erro do cliente.

5. 5xx: Erro do servidor.

3. Frase descritiva do código.

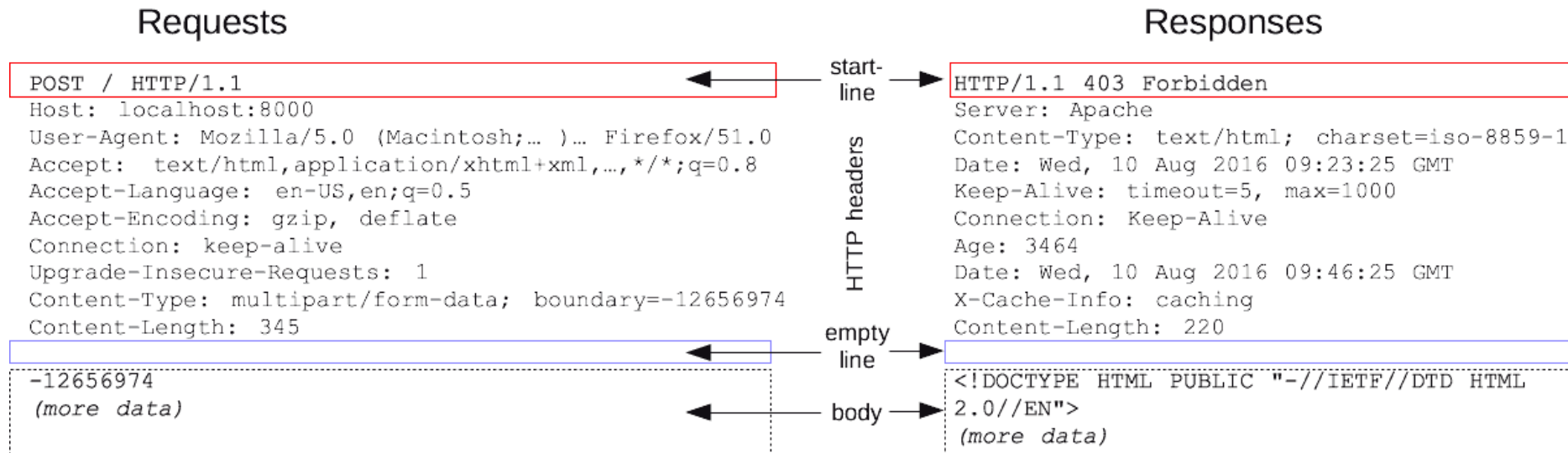
Exe.: **HTTP/1.0 404 Not Found**

# Código de status de resposta

| Status code | Standard text         | Meaning                                                                                                  |
|-------------|-----------------------|----------------------------------------------------------------------------------------------------------|
| 200         | OK                    | This indicates that the request was a success                                                            |
| 301         | Moved Permanently     | This indicates that the URL has been moved permanently and the link should be updated                    |
| 302         | Found                 | This indicates that the resource is temporarily located somewhere else, but the URL should still be used |
| 307         | Temporary Redirect    | This is similar to 302 , but the method used should not be changed, which may happen with 302            |
| 308         | Permanent Redirect    | This is similar to 301 , but the method used should not be changed, which may happen with 301            |
| 400         | Bad Request           | This indicates that request access was incorrect                                                         |
| 401         | Unauthorized          | This indicates that the resource is restricted often because the login attempt failed                    |
| 403         | Forbidden             | This indicates that access to the requested resource is forbidden                                        |
| 404         | Not Found             | This indicates that the resource is no longer available                                                  |
| 500         | Internal server error | This reflects some sort of error with the server                                                         |
| 502         | Bad Gateway           | This indicates that the gateway server received an invalid response from another server                  |
| 503         | Service Unavailable   | This indicates that the server is not available                                                          |
| 504         | Gateway Timeout       | This indicates that the gateway server did not receive a response from another server in a timely manner |

# Linhas de Cabeçalho

- As linhas de cabeçalho fornecem informações sobre a solicitação ou resposta, como o e-mail do remetente ou um identificador de aplicação.
- Formato: Identificador do cabeçalho: Valor.
- A linha de cabeçalho ocupa 1 linha



# Header Lines

POST / HTTP/1.1

Host: localhost:8000

User-Agent: Mozilla/5.0 (Macintosh;... )... Firefox/51.0

Accept: text/html,application/xhtml+xml,...,\*/\*;q=0.8

Accept-Language: en-US,en;q=0.5

Accept-Encoding: gzip, deflate

Connection: keep-alive

Upgrade-Insecure-Requests: 1

Content-Type: multipart/form-data; boundary=-12656974

Content-Length: 345

-12656974

(more data)

Request headers

General headers

Representation  
headers

# Header Lines

Uma lista de campos de cabeçalho e descrições pode ser encontrada em

[https://en.wikipedia.org/wiki/List\\_of\\_HTTP\\_header\\_fields](https://en.wikipedia.org/wiki/List_of_HTTP_header_fields).

Uma lista de strings de agentes pode ser encontrada em

<http://useragentstring.com/pages/useragentstring.php>.

O HTTP 1.0 define 16 cabeçalhos

(<http://www.w3.org/Protocols/HTTP/1.0/spec.html#HeaderFields>), enquanto o HTTP 1.1 possui 47 cabeçalhos

(<http://tools.ietf.org/html/rfc2616#section-14>).

# Corpo da Mensagem

- Este é o dado que constitui a mensagem. Ele pode ou não estar presente. Quando incluído, cabeçalhos como Content-Type e Content-Length são usados para fornecer mais informações.

**Content-type: text/html**

**Content-length: 105**

- O corpo da mensagem pode aparecer assim:

```
<html><h1>HTTPServer Home Page.... </h1><br><b>Welcome to the new and improved  
web server!</b><BR></html>
```

# No mundo real, existem dois tipos de servidores Web/HTTP

- **Modelo de I/O: Bloqueante**
  - Tomcat
  - Django(Python)
  - Flask(Python)
  - FPM (FastCGI Process Manager)
  - **Puma** para Ruby on Rails
- **Modelo de I/O: Não-Bloqueante**
  - FastAPI(Python)
  - Sanic(Python)
  - Tornado(Python)
  - Node.js
  - Netty
  - ReactPHP

# Modelo de I/O: Bloqueante

- Nessa arquitetura, o servidor aloca uma **nova thread** para cada conexão/requisição de cliente. Essa thread fica dedicada e "bloqueada" até que a requisição seja completamente processada e a resposta seja enviada.
- **Vantagens:**
  - **Simples:** Fácil de entender e de programar.
  - **Ideal para CPU Intensiva:** Eficiente para requisições que exigem muito processamento do servidor (cálculos pesados, por exemplo).
- **Desvantagens:**
  - **Escalabilidade limitada:** O servidor pode ficar sem threads, sobrecarregado por muitas conexões simultâneas.
  - **Alto consumo de recursos:** Threads ociosas, esperando por I/O, desperdiçam memória e recursos da CPU.

# Modelo de I/O: Não-Bloqueante

- Aqui, uma única thread (ou um pequeno pool de threads-Event Loop) gerencia **todas as conexões** de forma assíncrona. O servidor não espera por I/O; ele registra a conexão, delega a tarefa e continua processando outras requisições.
- **Vantagens:**
  - **Alta escalabilidade:** Consegue lidar com milhares de conexões simultâneas com baixo consumo de recursos.
  - **Eficiente:** Não há threads ociosas esperando por dados.
- **Desvantagens:**
  - **Complexo:** Mais difícil de programar e depurar.
  - **Sensível a CPU:** Uma única operação que usa muita CPU pode bloquear o servidor inteiro.

# Exemplo de Servidor de I/O: Bloqueante

```
public class WebServer {

    public WebServer() {
        System.out.println("Webserver Started");
        try (ServerSocket serverSocket = new ServerSocket(8080)) {
            while (true) {
                System.out.println("Waiting for client request");
                Socket remote = serverSocket.accept();
                System.out.println("Connection made");
                new Thread(new ClientHandler(remote)).start();
            }
        } catch (IOException ex) {
            ex.printStackTrace();
        }
    }

    public static void main(String args[]) {
        new WebServer();
    }
}
```

# Java Socket Support for HTTP (Server Side)

```
public class ClientHandler implements Runnable {  
    private final Socket socket;  
    public ClientHandler(Socket socket) {  
        this.socket = socket;  
    }  
    @Override  
    public void run() {  
        System.out.println("\nClientHandler Started for " +  
            this.socket);  
        handleRequest(this.socket);  
        System.out.println("ClientHandler Terminated for "  
            + this.socket + "\n");  
    }  
}
```

O método `handleRequest` utiliza fluxos de entrada e saída para se comunicar com o servidor. Além disso, ele determina qual solicitação foi feita e, em seguida, processa essa solicitação.

# Java Socket Support for HTTP (Server Side)

```
public void handleRequest(Socket socket) {
    try (BufferedReader in = new BufferedReader(
        new InputStreamReader(socket.getInputStream()));) {
        String headerLine = in.readLine();
        StringTokenizer tokenizer = new StringTokenizer(headerLine);
        String httpMethod = tokenizer.nextToken();

        if (httpMethod.equals("GET")) {
            System.out.println("Get method processed");
            String httpQueryString = tokenizer.nextToken();
            StringBuilder responseBuffer = new StringBuilder();
            responseBuffer
                .append("<html><h1>WebServer Home Page.... </h1><br>")
                .append("<b>Bem vindo ao Meu web server!</b><BR>")
                .append("</html>");
            sendResponse(socket, 200, responseBuffer.toString());
        } else {
            System.out.println("The HTTP method is not recognized");
            sendResponse(socket, 405, "Method Not Allowed");
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

# Java Socket Support for HTTP (Server Side)

```
public void sendResponse(Socket socket, int statusCode, String responseString) {
    String statusLine;
    String serverHeader = "Server: WebServer\r\n";
    String contentTypeHeader = "Content-Type: text/html\r\n";

    try (DataOutputStream out = new DataOutputStream(socket.getOutputStream())) {
        if (statusCode == 200) {
            statusLine = "HTTP/1.0 200 OK" + "\r\n";
            String contentLengthHeader = "Content-Length: " + responseString.length() + "\r\n";

            out.writeBytes(statusLine);
            out.writeBytes(serverHeader);
            out.writeBytes(contentTypeHeader);
            out.writeBytes(contentLengthHeader);
            out.writeBytes("\r\n");
            out.writeBytes(responseString);
        } else if (statusCode == 405) {
            statusLine = "HTTP/1.0 405 Method Not Allowed" + "\r\n";
            out.writeBytes(statusLine);
            out.writeBytes("\r\n");
        } else {
            statusLine = "HTTP/1.0 404 Not Found" + "\r\n";
            out.writeBytes(statusLine);
            out.writeBytes("\r\n");
        }
        out.close();
    } catch (IOException ex) {
        ex.printStackTrace();
    }
}
```

# Java Socket Support for HTTP (Client Side)

```
public class HTTPClient {

    public HTTPClient() {
        System.out.println("HTTP Client Started");
        try {
            InetAddress serverInetAddress =
                InetAddress.getByName("127.0.0.1");
            Socket connection = new Socket(serverInetAddress, 80);

            try (OutputStream out = connection.getOutputStream();
                BufferedReader in =
                    new BufferedReader(new
                        InputStreamReader(
                            connection.getInputStream()))) {
                sendGet(out);
                System.out.println(getResponse(in));
            }
        } catch (IOException ex) {
            ex.printStackTrace();
        }
    }

    ...

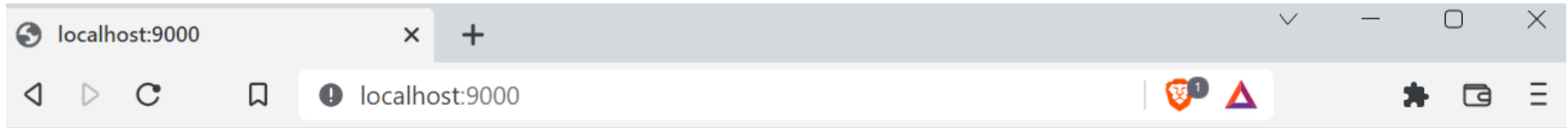
    public static void main(String[] args) {
        new HTTPClient();
    }
}
```

# Java Socket Support for HTTP (Client Side)

```
private void sendGet(OutputStream out) {  
    try {  
        out.write("GET /default\r\n".getBytes());  
        out.write("User-Agent: Mozilla/5.0\r\n".getBytes());  
    } catch (IOException ex) {  
        ex.printStackTrace();  
    }  
}
```

```
private String getResponse(BufferedReader in) {  
    try {  
        String inputLine;  
        StringBuilder response = new StringBuilder();  
        while ((inputLine = in.readLine()) != null) {  
            response.append(inputLine).append("\n");  
        }  
        return response.toString();  
    } catch (IOException ex) {  
        ex.printStackTrace();  
    }  
    return "";  
}
```

# Testando seu servidor



## WebServer Home Page....

Welcome to my web server!

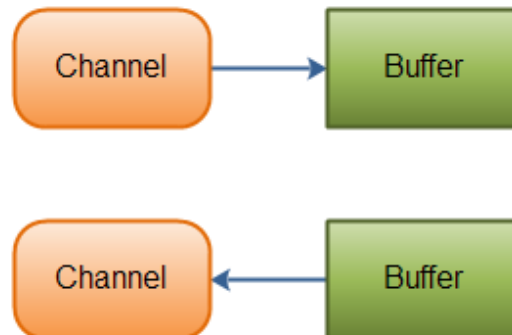
# Modelo de I/O: Não-Bloqueante com Java NIO.2

O pacote **java.nio** foi introduzido no Java 1.4 e atualizado no Java 1.7 (NIO.2) com operações de arquivo aprimoradas e um **ASynchronousSocketChannel**.

- **Buffer** – para ler blocos de dados de uma vez
  - **Channel** – para comunicar-se com o mundo externo
  - **Selector** – para permitir multiplexação em um **SelectableChannel** e fornecer acesso a qualquer **Channel** que esteja pronto para E/S
- Modo não bloqueante – para ler o que estiver pronto

# Channels and Buffers

- Normalmente, toda operação de I/O no NIO começa com um **Channel**.
- Um **Channel** é como um stream. A partir do **Channel**, os dados podem ser lidos em um **Buffer**. Os dados também podem ser escritos de um **Buffer** em um **Channel**.



# Channels types

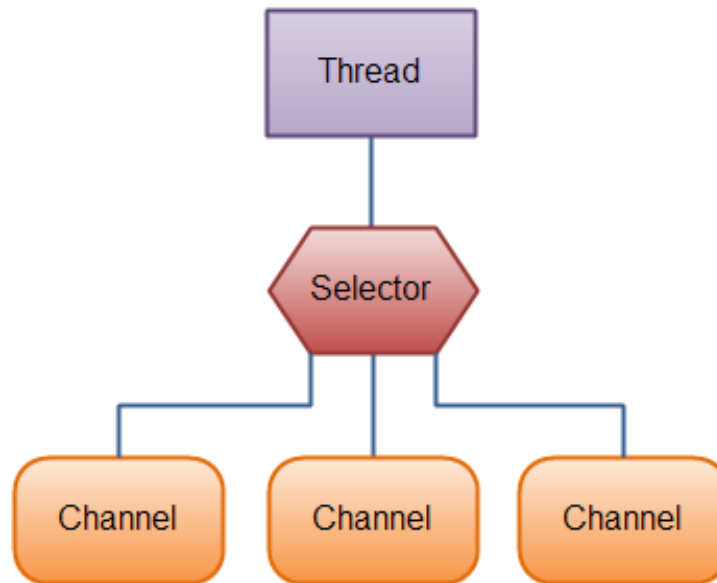
- Existem vários tipos de **Channel**. Aqui está uma lista das principais implementações de **Channel** no Java NIO:
  - **FileChannel** lê dados de arquivos e grava dados em arquivos;
  - **DatagramChannel** pode ler e escrever dados pela rede via UDP;
  - **SocketChannel** pode ler e escrever dados pela rede via TCP;
  - **ServerSocketChannel** permite que você escute conexões TCP recebidas, como faz um servidor web. Para cada conexão recebida, um **SocketChannel** é criado.

# Buffer types

- Aqui está uma lista das implementações principais de **Buffer** no Java NIO:
  - **ByteBuffer**
  - **CharBuffer**
  - **DoubleBuffer**
  - **FloatBuffer**
  - **IntBuffer**
  - **LongBuffer**
  - **ShortBuffer**

# Selectors

- Um **Selector** permite que uma única thread manipule vários **Channels**. Isso é útil se sua aplicação tiver muitas conexões (**Channels**) abertas, mas com pouco tráfego em cada conexão.



# TCP com NIO

- Um **ServerSocketChannel** do Java NIO é um canal que pode escutar conexões TCP recebidas, assim como um **ServerSocket** na rede padrão do Java. A classe **ServerSocketChannel** está localizada no pacote **java.nio.channels**.

Run main | Debug main

```
public static void main(String args[]) throws IOException {  
  
    System.out.println("Meu NIO Webserver Started");  
    ServerSocketChannel serversocketchannel = ServerSocketChannel.open();  
    serversocketchannel.socket().bind(new InetSocketAddress(9999));  
    while(true){  
        SocketChannel socketChannel = serversocketchannel.accept();  
    }  
}
```

# Modelo de I/O: Não-Bloqueante

- Um **ServerSocketChannel** pode ser configurado no modo não bloqueante. No modo não bloqueante, o método **accept()** retorna imediatamente e pode, portanto, retornar **null** se nenhuma conexão recebida tiver chegado. Portanto, você precisará verificar se o **SocketChannel** retornado é **null**.

```
public static void main(String args[]) throws IOException {
    System.out.println("Meu NIO Webserver Started");
    ServerSocketChannel serversocketchannel = ServerSocketChannel.open();
    serversocketchannel.socket().bind(new InetSocketAddress(9999));
    serversocketchannel.configureBlocking(false);
    while(true){
        SocketChannel socketChannel = serversocketchannel.accept();
        if(socketChannel != null){
            }
        }
    }
}
```

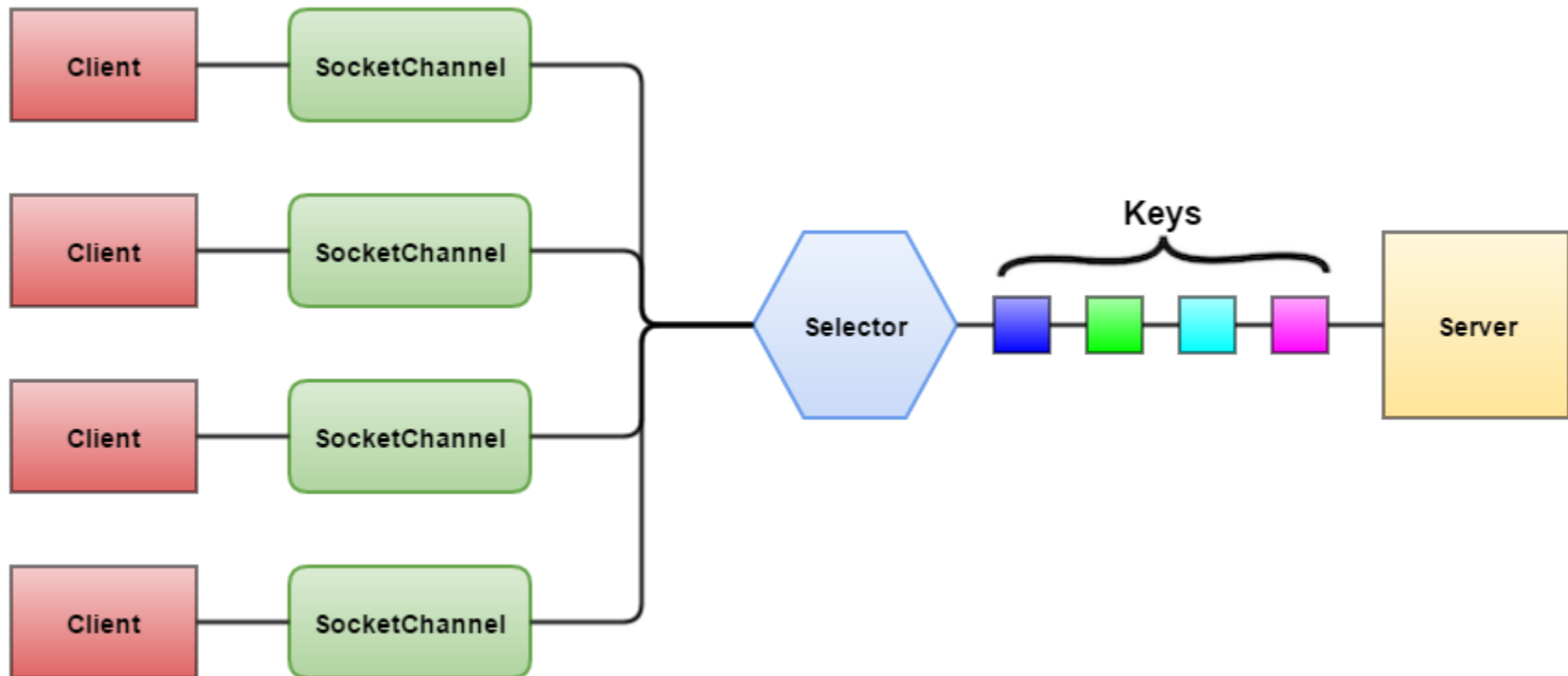
# NIO SocketChannel

- Um **SocketChannel** do Java NIO é um canal conectado a um socket de rede TCP. Ele é o equivalente no Java NIO aos **Sockets** da rede Java. Existem duas maneiras de criar um **SocketChannel**:
  1. Você abre um **SocketChannel** e se conecta a um servidor em algum lugar da internet.
  2. Um **SocketChannel** pode ser criado quando uma conexão recebida chega a um **ServerSocketChannel**

# Usando *Selector* para Melhorar Escalabilidade

- Um seletor é usado em uma aplicação NIO e permite que uma thread manipule vários canais.
- O seletor coordena vários canais e seus eventos.
- Ele identifica aqueles canais que estão prontos para processamento.
- Se usássemos uma thread por canal, nos encontraríamos alternando entre threads com frequência. Esse processo de troca pode ser caro(10.000 ns). Usar uma única thread para manipular múltiplos canais evita parte dessa sobrecarga.

# Usando *Selector* para Melhorar Escalabilidade



## Criar e registrar eventos

Uma vez que temos um seletor, chamaremos o método `register()` disponível nos objetos de canal (`DatagramChannel`, `ServerSocketChannel`, `ServerSocket`). Isso é feito para registrar o interesse nos eventos de I/O que estão ocorrendo nesses objetos.

```
Selector selector = Selector.open();  
channel1.register(selector, SelectionKey.OP_ACCEPT);  
channel2.register (selector, SelectionKey.OP_CONNECT);  
// registrar interesse em mais de um seletor  
channel4.register(selector, SelectionKey.OP_READ | SelectionKey.OP_WRITE);
```

# Obtendo os canais prontos

- Uma vez que você registrou um ou mais canais em um Seletor, pode chamar um dos métodos `select()`. Esses métodos retornam os canais que estão "prontos" para os eventos do seu interesse (conectar, aceitar, ler ou escrever). Em outras palavras, se você está interessado em canais que estão prontos para leitura, receberá os canais que estão prontos para leitura dos métodos `select()`.
- O `select()` bloqueia até que pelo menos um canal esteja pronto para os eventos que você registrou.
- O `select(long timeout)` faz o mesmo que o `select()`, exceto que bloqueia por um máximo de timeout milissegundos (o parâmetro).
- O `selectNow()` não bloqueia de forma alguma. Ele retorna imediatamente com os canais que estão prontos.

## Passando por todos os canais prontos

- Uma vez que você chamou um dos métodos `select()` e seu valor de retorno indicou que um ou mais canais estão prontos, você pode acessar os canais prontos através do "conjunto de chaves selecionadas", chamando o método `selectedKeys()` do seletor.

```
int readyChannels = selector.select();
```

```
Set<SelectionKey> selectedKeys = selector.selectedKeys();
```

```
Iterator<SelectionKey> i= selectedKeys.iterator();
```

# Passando por todos os canais prontos

```
while(i.hasNext()) {  
    SelectionKey key = i.next();  
  
    if (key.isAcceptable()) {  
        processAcceptable(key); // connection accepted  
    } else if (key.isConnectable()) {  
        processConnectable(key); // connection established  
    } else if (key.isReadable()) {  
        processReadable(key); // ready for reading  
    } else if (key.isWritable()) {  
        processWritable(key); // ready for writing  
    }  
}
```

# Código Servidor

```
public static void main(String args[]) throws IOException {
    System.out.println(x:"Meu NIO Webserver Started");
    selector = Selector.open();

    ServerSocketChannel serversocketchannel = ServerSocketChannel.open();
    serversocketchannel.socket().bind(new InetSocketAddress(port:9999));
    serversocketchannel.configureBlocking(block:false);
    serversocketchannel.register(selector, SelectionKey.OP_ACCEPT);

    while (true) {
        selector.select();
        Set<SelectionKey> selectedKeys = selector.selectedKeys();
        Iterator<SelectionKey> i = selectedKeys.iterator();

        while (i.hasNext()) {
            SelectionKey key = i.next();

            if (key.isAcceptable()) {
                processAcceptEvent(serversocketchannel, key);
            } else if (key.isReadable()) {
                processReadEvent(key);
            }
            i.remove();
        }
    }
}
```

# Código Servidor

```
private static void processAcceptEvent(ServerSocketChannel mySocket,  
    SelectionKey key) throws IOException {  
    System.out.println(x:"Conexão aceita...");  
    // aceita a conexão do cliente e faz ele não-bloqueante  
    SocketChannel myClient = mySocket.accept();  
    myClient.configureBlocking(block:false);  
    // Registra o canal do cliente com o seletor para ler  
    myClient.register(selector, SelectionKey.OP_READ);  
}
```

# Código para envio da resposta

```
private static void processReadEvent(SelectionKey key)
    throws IOException {
    System.out.println(x:"Processando evento de leitura...");
    SocketChannel myClient = (SocketChannel) key.channel();
    ByteBuffer myBuffer = ByteBuffer.allocate(BUFFER_SIZE);
    myClient.read(myBuffer);
    String headerLine = new String(myBuffer.array()).trim();
    if (headerLine.length() > 0) {
        StringTokenizer tokenizer = new StringTokenizer(headerLine);
        String httpMethod = tokenizer.nextToken();
        if (httpMethod.equals(anObject:"GET")) {
            System.out.println(x:"Processando o método GET");
            StringBuilder responseBuffer = new StringBuilder();
            responseBuffer
                .append(str:"<html><h1>WebServer Home Page.... </h1><br>")
                .append(str:"<b>Bem vindo ao Meu web server! </b><BR>")
                .append(str:"</html>");
            sendResponse(myClient, statusCode:200, responseBuffer.toString());
        } else {
            System.out.println(x:"The HTTP method is not recognized");
            sendResponse(myClient, statusCode:405, responseString:"Method Not Allowed");
        }
    }
}
```

# Código montagem pacote HTTP

```
public static void sendResponse(SocketChannel socket, int statusCode, String responseString) {
    String statusLine;
    String serverHeader = "Server: WebServer\r\n";
    String contentTypeHeader = "Content-Type: text/html\r\n";

    try {
        StringBuilder response = new StringBuilder();

        if (statusCode == 200) {
            statusLine = "HTTP/1.1 200 OK\r\n";
            String contentLengthHeader = "Content-Length: " + responseString.getBytes().length + "\r\n";
            response.append(statusLine);
            response.append(serverHeader);
            response.append(contentTypeHeader);
            response.append(contentLengthHeader);
            response.append(str:"\r\n");
            response.append(responseString);
        } else if (statusCode == 405) {
            statusLine = "HTTP/1.1 405 Method Not Allowed\r\n";
            response.append(statusLine).append(str:"\r\n");
        } else {
            statusLine = "HTTP/1.1 404 Not Found\r\n";
            response.append(statusLine).append(str:"\r\n");
        }
        // Converte para bytes e escreve no canal
        ByteBuffer buffer = ByteBuffer.wrap(response.toString().getBytes());
        while (buffer.hasRemaining()) {
            socket.write(buffer);
        }
        socket.close(); // fecha a conexão após resposta simples
    } catch (IOException ex) {
        ex.printStackTrace();
    }
}
```

# Representational State Transfer (REST)

UNIVERSITY OF CALIFORNIA,  
IRVINE

Architectural Styles and the Design of Network-based Software Architectures

DISSERTATION

submitted in partial satisfaction of the requirements for the degree of

DOCTOR OF PHILOSOPHY

in Information and Computer Science

by

Roy Thomas Fielding

Dissertation Committee:  
Professor Richard N. Taylor, Chair  
Professor Mark S. Ackerman  
Professor David S. Rosenblum

2000

# Representational State Transfer (REST)

- Pense em REST como um conjunto de "melhores práticas" ou restrições que, quando aplicadas a um sistema, garantem que ele seja **escalável, simples e uniforme**. Em vez de ser um protocolo rígido, REST é um estilo arquitetural que usa o HTTP da maneira como ele foi projetado para ser usado.
- O principal conceito é que tudo no sistema é um **recurso**. Cada recurso é identificado por uma **URL única** (Uniform Resource Locator). Por exemplo:
  - /usuarios
  - /usuarios/123
  - /produtos
  - /produtos/456

# Os Princípios Fundamentais do REST

- O estilo REST se baseia em seis restrições principais, mas as mais importantes para entender na prática são:
  - **Interface Uniforme:** A maneira como você interage com a API é sempre a mesma. Isso significa usar os métodos HTTP padrão para realizar ações sobre os recursos.
  - **Stateless (Sem Estado):** O servidor não armazena nenhuma informação sobre as requisições anteriores do cliente. Cada requisição deve conter todas as informações necessárias para que o servidor a processe. Isso torna a API mais escalável.
  - **Client-Server:** O cliente e o servidor são separados. Isso permite que cada um evolua de forma independente. O cliente não precisa saber como o servidor armazena os dados, apenas como interagir com a API.
  - **Recursos e Métodos HTTP:** Esta é a parte mais prática para quem está programando. A arquitetura REST associa as operações CRUD (Create, Read, Update, Delete) com os verbos (métodos) do HTTP.

# Os Princípios Fundamentais do REST

| Ação CRUD                    | Método HTTP     | URL de Exemplo | Descrição                                                             |
|------------------------------|-----------------|----------------|-----------------------------------------------------------------------|
| <b>CREATE</b> (Criar)        | POST            | /usuarios      | Cria um novo recurso (um novo usuário) no servidor.                   |
| <b>READ</b> (Ler)            | GET             | /usuarios      | Obtém uma lista de todos os recursos.                                 |
| <b>READ</b> (Ler)            | GET             | /usuarios/123  | Obtém um recurso específico (o usuário com ID 123).                   |
| <b>UPDATE</b><br>(Atualizar) | PUT ou<br>PATCH | /usuarios/123  | PUT atualiza o recurso inteiro. PATCH atualiza apenas uma parte dele. |
| <b>DELETE</b><br>(Deletar)   | DELETE          | /usuarios/123  | Deleta um recurso específico.                                         |

# Por que REST é tão popular?

- **Simplicidade:** Usa as funcionalidades do HTTP que já conhecemos. Não exige bibliotecas ou protocolos complexos.
- **Performance:** O uso inteligente de métodos HTTP e a natureza sem estado ajudam na performance.
- **Escalabilidade:** A falta de estado no servidor facilita a distribuição da carga de trabalho entre vários servidores.
- **Flexibilidade:** Não impõe tecnologias específicas. Uma API RESTful pode ser consumida por um site em JavaScript, um aplicativo móvel em Java, ou qualquer outra plataforma.

# Popularidade do REST com HTTP1.1

## ADOPTION & USAGE

### HTTP Versions

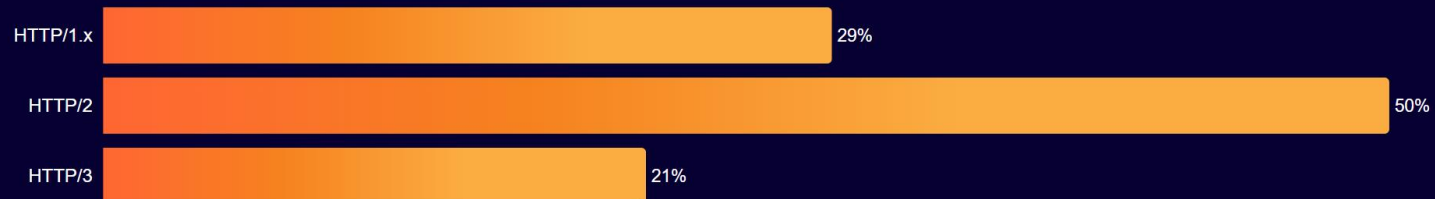
21% of traffic uses HTTP/3

HTTP (HyperText Transfer Protocol) is the core protocol that the web relies upon. **HTTP/1.0** was first standardized in 1996, **HTTP/1.1** in 1999, and **HTTP/2** in 2015. The most recent version, **HTTP/3**, was completed in 2024, and runs on top of QUIC, a new transport protocol. Using **QUIC** allows it to mitigate the effects of packet loss, establish connections more quickly, and provide encryption by default.

Here we look at the distribution of HTTP versions across request traffic seen by Cloudflare globally during 2025.

Share   
Copy link   
Compare 

Summary Ranking Map



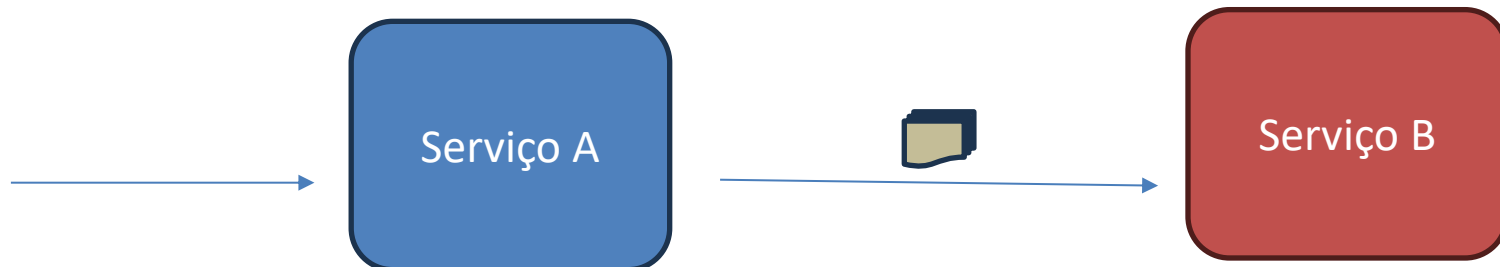
<https://radar.cloudflare.com/year-in-review/2025#http-versions>

# Essa popularidade é acompanhada por limitações

- O HTTP/1.1 foi introduzido em 1997
- **Características**
  - Conexões persistentes (Keep-Alive)
  - Cabeçalhos textuais
  - Problema de *Head-of-Line blocking*
- Limitação: múltiplos requests precisam de várias conexões TCP

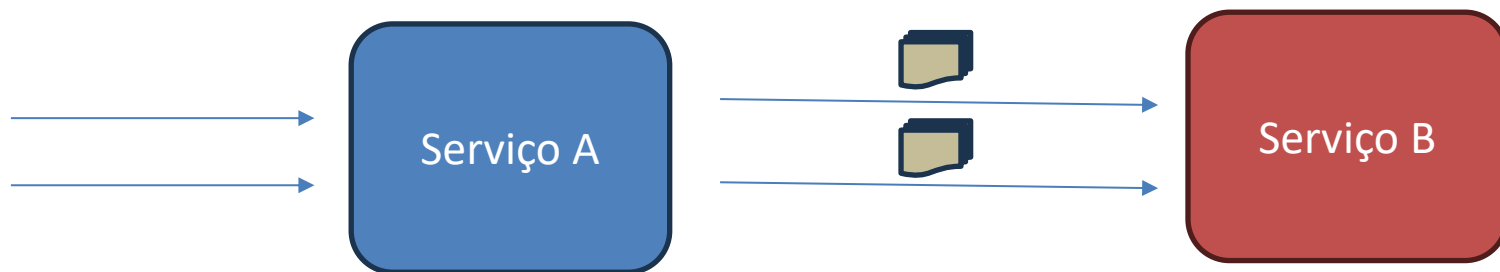
# Conexão interserviços

- O problema ocorre quando o Serviço A recebe muitas requisições de serviço, ele precisa criar uma nova conexão com o Serviço B, isso envolve passar por todo o processo de *3 way handshake*



# Conexão interserviços

- Uma conexão TCP só pode tratar uma requisição por vez. Se precisar de várias requisições simultâneas → abrir múltiplas conexões (6 por host em média nos navegadores). Sofre com o problema de **Head-of-Line Blocking** (se a primeira request demora, as seguintes ficam esperando).

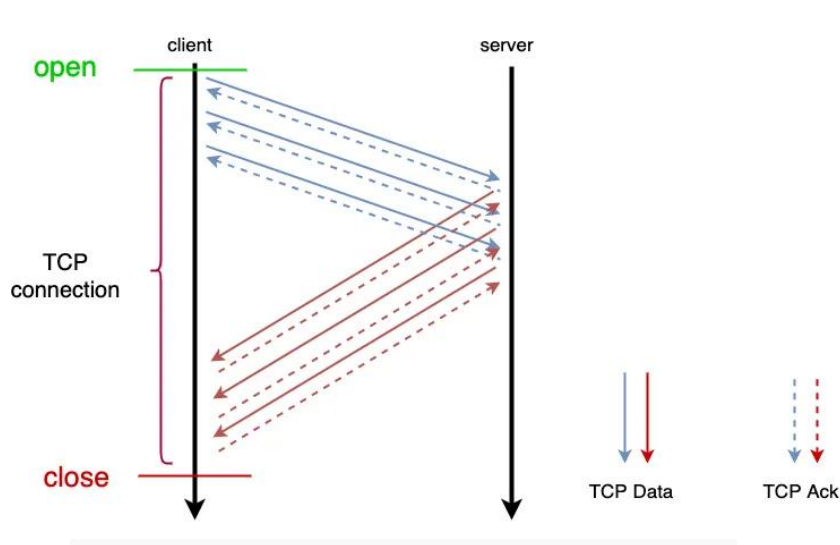


# Head-of-Line (HOL) Blocking

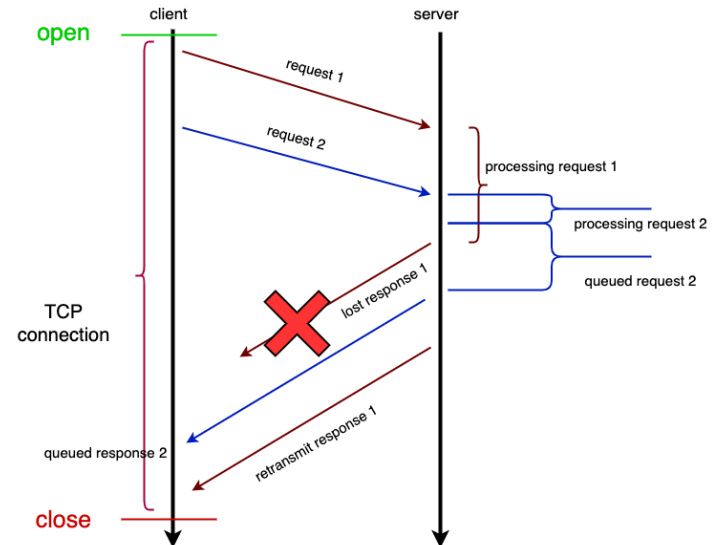
- O **Head-of-Line (HOL) Blocking** no **HTTP/1.1** é um problema de **bloqueio na fila de requisições**, no qual **uma requisição lenta impede que as próximas sejam processadas**, mesmo que elas possam ser atendidas rapidamente.
- Esse problema ocorre principalmente porque o **HTTP/1.1** usa **uma conexão TCP sequencial para múltiplas requisições**.
- Exemplo: No **HTTP/1.1**, quando o navegador precisa carregar uma página, ele faz várias requisições: HTML, CSS, JavaScript ,imagens, fontes

# Head-of-Line (HOL) Blocking

- O **Head-of-Line (HOL) Blocking** no **HTTP/1.1** é um problema de **bloqueio na fila de requisições**, no qual **uma requisição lenta impede que as próximas sejam processadas**, mesmo que elas possam ser atendidas



Requisição 1 -> resposta lenta  
Requisição 2 -> BLOQUEADA  
Requisição 3 -> BLOQUEADA

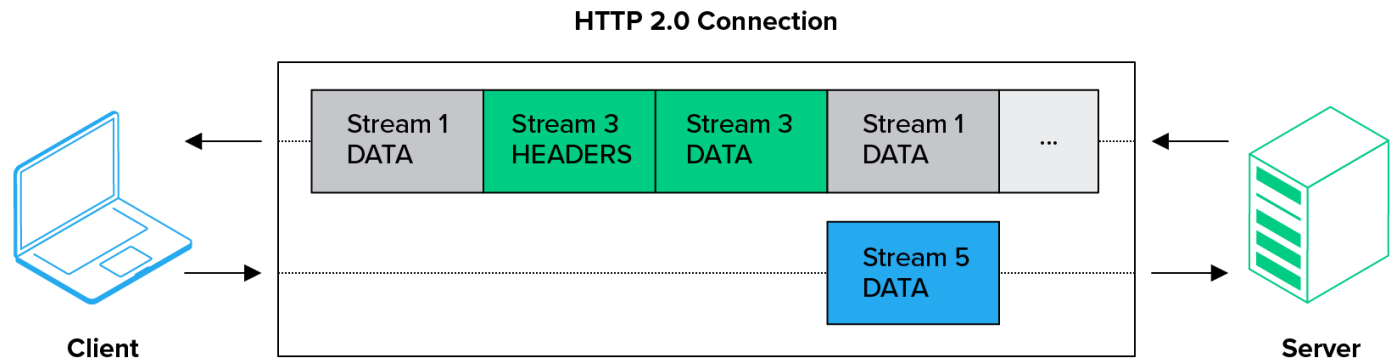


# HTTP/2

- **HTTP/2** foi lançado em 2015
- **Novidades**
  - Binário, não mais textual
  - Multiplexação em uma única conexão TCP
  - Compressão de cabeçalhos (HPACK)
  - Server Push
- Muito mais eficiente que HTTP/1.1

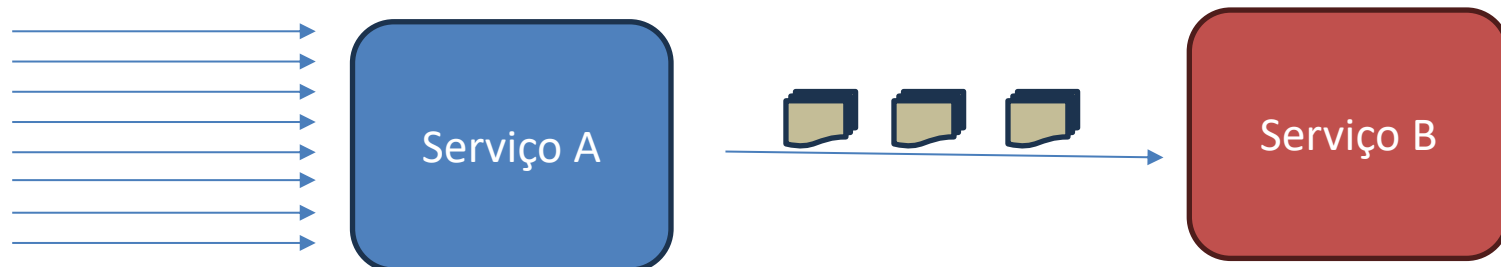
# HTTP/2 - Binário, não mais textual

- Substitui a representação textual por **frames binários**.
- Toda comunicação é feita em **streams de frames** (cada um identificado por um ID).
- Estrutura rígida → menos ambiguidade, parsing muito mais rápido.
- Cabeçalhos e corpo são separados em diferentes frames.
- **Principais tipos de frame:**
  - HEADERS → metadados da requisição/resposta
  - DATA → conteúdo (HTML, JSON, etc.)
  - SETTINGS → parâmetros de conexão
  - Etc.



# HTTP/2 - Multiplexação em uma única conexão TCP

- Permite múltiplos streams paralelos na mesma conexão TCP.
- Cada stream tem um identificador → servidor e cliente podem enviar várias respostas simultaneamente.
- Evita o bloqueio de requisições grandes sobre as pequenas.



# HTTP/2 - Multiplexação em uma única conexão TCP

HTTP 1.1

**3** TCP CONNECTIONS



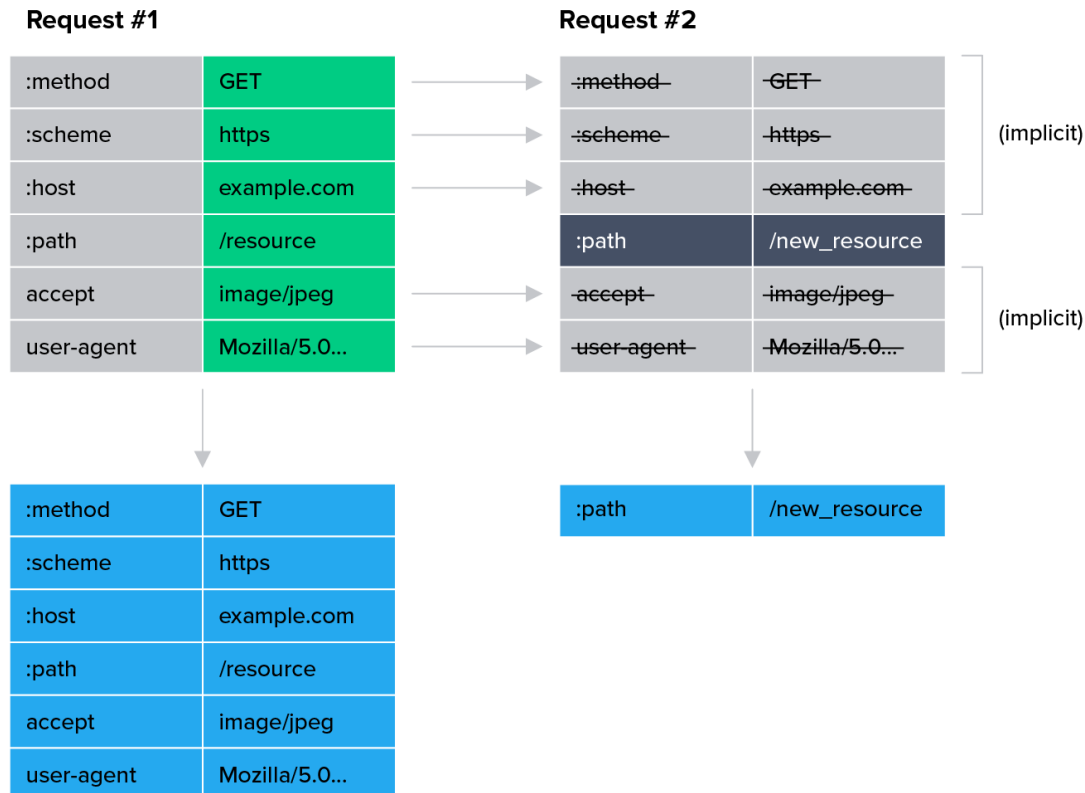
HTTP/2

**1** TCP CONNECTION



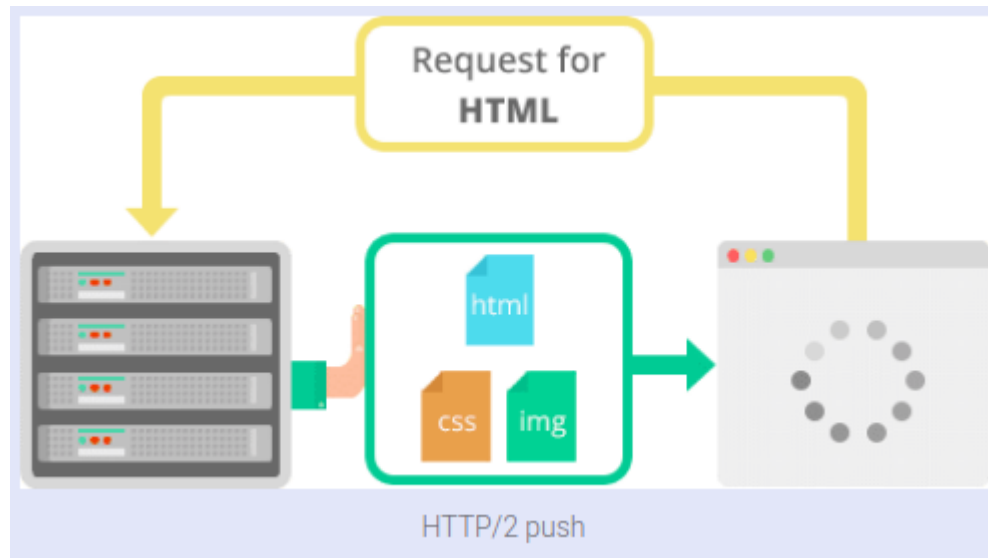
# HTTP/2 - Compressão de cabeçalhos

- Introduziu **HPACK**:
  - Usa **dicionários dinâmicos/estáticos** para evitar repetição.
  - Substitui strings por índices → reduz drasticamente o tamanho.



# HTTP/2 - Server Push

- Servidor pode “empurrar” recursos antes mesmo do cliente solicitar.
- Exemplo: ao pedir index.html, o servidor já envia style.css e script.js.
- Reduz tempo de carregamento de páginas.



# Motivos para a criação do HTTP3

- A principal motivação para criar o **HTTP/3** foi **resolver limitações estruturais do transporte usado pelo HTTP/2**, especialmente aquelas causadas pelo uso do **TCP**.
- Embora o HTTP/2 tenha melhorado bastante o desempenho da Web, alguns problemas **não podiam ser resolvidos no nível da aplicação**, porque estavam no protocolo de transporte. Por isso surgiu o HTTP/3, que utiliza **QUIC** sobre **UDP**.

# HTTP/3

- Padrão recente (2022)
- Baseado em QUIC (sobre UDP)
- Vantagens:
  - Reduz latência de handshake
  - Sem Head-of-Line blocking de TCP
  - Mais rápido em redes móveis (perdas toleradas)

# Eliminar o Head-of-Line Blocking no transporte

- O **HTTP/2** introduziu **multiplexação de streams**, permitindo múltiplas requisições em uma única conexão.
- Porém, ele ainda depende do **TCP**, que entrega dados **estritamente em ordem**.
- Se um pacote se perde:

Pacote 1 ✓  
Pacote 2 ✗ perdido  
Pacote 3 ✓
- O TCP **não entrega o pacote 3 até que o 2 seja retransmitido**.
- Consequência:
  - todos os streams HTTP/2 ficam bloqueados
  - Isso é chamado de **Head-of-Line Blocking no transporte**.

# Reduzir latência de conexão

- No **HTTP/2**, a conexão exige dois handshakes:

- 1 handshake TCP
- 1 handshake TLS

```
Cliente -> SYN
Servidor -> SYN-ACK
Cliente -> ACK
+
TLS handshake
```

- O **QUIC** integra **transporte + criptografia**.

- Resultado:
  - **1 RTT handshake**
  - **0 RTT reconnection** em conexões repetidas
- Isso reduz bastante a latência inicial de páginas.

# Melhor suporte a mobilidade de rede

- Conexões TCP são identificadas por:
  - IP + porta origem
  - IP + porta destino
- Se o IP do cliente muda (ex: troca de Wi-Fi para 4G) a conexão TCP **quebra**. Isso afeta diretamente o **HTTP/2**.
- O **QUIC** usa um **Connection ID**, permitindo:
  - migração de rede
  - continuidade da conexão

# Evolução mais rápida do protocolo

- Modificar o **TCP** é extremamente difícil porque:
  - está implementado no **kernel do sistema operacional**
  - mudanças exigem **deploy global**
- O **QUIC** roda em **user space**, permitindo:
  - atualização via browser
  - deploy rápido
  - experimentação
- Isso acelera muito a evolução da Web.

# Resumo

| Característica     | HTTP/1.1 | HTTP/2    | HTTP/3         |
|--------------------|----------|-----------|----------------|
| Ano                | 1997     | 2015      | 2022           |
| Base               | TCP      | TCP       | UDP (QUIC)     |
| Multiplexação      | ✗        | ✓         | ✓              |
| Formato            | Texto    | Binário   | Binário        |
| Compressão headers | ✗        | ✓ (HPACK) | ✓ (QPACK)      |
| Server Push        | ✗        | ✓         | ⚠ (em revisão) |

# HTTP/3 Support in JDK 26

- <https://inside.java/2025/10/22/http3-support/>

# HTTP Sampler

## HTTP Request

Name:

Comments:



Basic Advanced

### Web Server

Protocol [http]:  Server Name or IP:  Port Number:

### HTTP Request

Path:  Content encoding:

☐ Redirect Automatically ☐ Follow Redirects ☐ Use KeepAlive ☐ Use multipart/form-data ☐ Browser-compatible headers



Parameters Body Data Files Upload

### Send Parameters With the Request:

| Name: | Value | URL Encode? | Content-Type | Include Equals? |
|-------|-------|-------------|--------------|-----------------|
|-------|-------|-------------|--------------|-----------------|

Detail

Add

Add from Clipboard

Delete

Up

Down